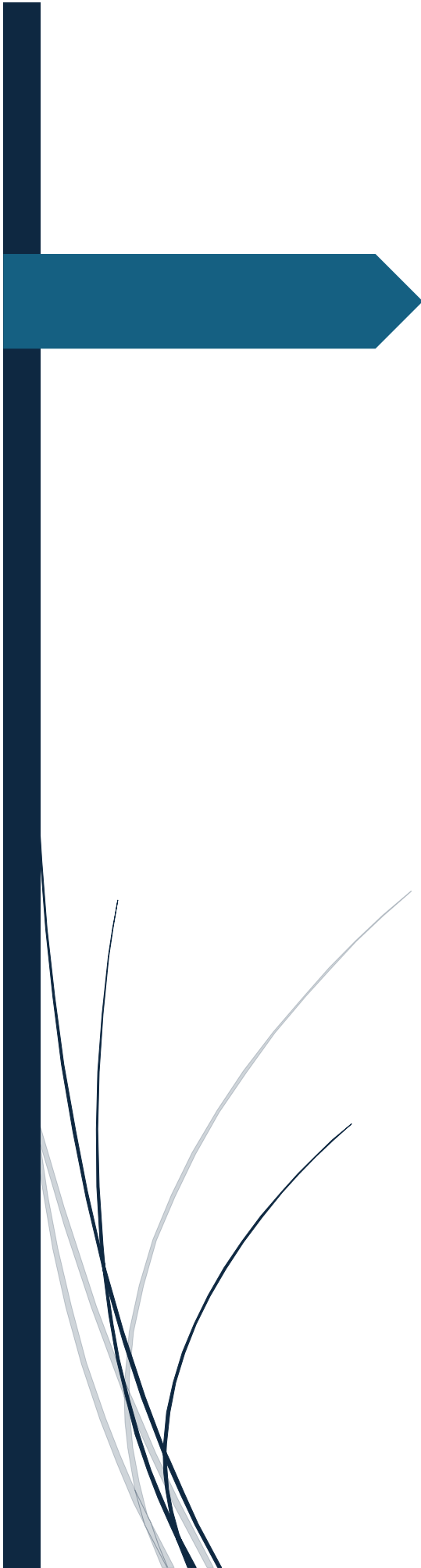




Phase 2: MNIST Multi-classifier Technical Report

Loay Khaled Mohammed Abdelmotelb	23P0340
Abdelrahman Wael Maher	23P0155
Seif Amjad Sobeih	23P0145
Ahmed Hossam Abdallah	23P0109
Youssef Farid Haddad	23P0113

CONTENTS

Model: Multinomial logistic Regression	4
Data Preparation	4
Model Architecture	4
Optimization Formulation	4
Custom Metric Implementations	5
Accuracy	5
ROC-AUC (One-vs-Rest)	5
Diagnostics and Analysis	5
K-Fold Cross-Validation.....	5
Learning Curves	5
Bias-Variance Trade-off	6
Results.....	6
Model: K-Nearest Neighbours	7
Mathmetical formulation:.....	7
Mathematical Formulation of Hyperparameter Tuning.....	7
Evaluation Metrics (Implemented from Scratch)	8
Ensemble Methods	8
Implementation: Feature Representations	8
Hyperparameter Tuning 5-Fold Stratified Grid Search	8
Experimental Evaluation	9
Improvement Strategies	9
BiasVariance Analysis	10
Overfitting/Underfitting diagnosis and learning curves	10
Model: Kernel SVM	11
Data Preparation	11
Model Optimization Formulation	11
Class Imbalance Strategies.....	12
Variant A: Class Weights	12
Variant B: Random Undersampling	12
Variant C: Synthetic Minority Oversampling Technique (SMOTE)	12
Results and Diagnostics.....	12
One-vs-All Optimization Architecture.....	12
Hyperparameter Tuning with Cross-Validation.....	13
Bias-Variance Analysis.....	13
Overfitting/Underfitting diagnosis and learning curves	14
Results.....	14
Model: Multiclass Linear SVM	15
Data Preparation	15

Mathematical Formulation	15
Optimization with Adam	15
Feature Pipelines	16
Hyperparameter Tuning with Cross-Validation.....	16
Bias-Variance Analysis.....	16
Ensemble Baselines	16
Results.....	17
Github link	17

MODEL: MULTINOMIAL LOGISTIC REGRESSION

DATA PREPARATION

The MNIST dataset (70,000 images of handwritten digits, 784 pixels each) was processed through the following pipeline:

- **Train/Test Split:** 60,000 training samples and 10,000 test samples.
- **Standardization:** Features were scaled to zero mean and unit variance using.
- **PCA:** Dimensionality reduction from 784 to 50 principal components.
- **Polynomial Expansion:** A degree-2 polynomial expansion was applied to the 50 PCA components, generating approximately 1,300 non-linear features.
- **One-Hot Encoding:** Labels were converted to a $N \times 10$ one-hot matrix.

MODEL ARCHITECTURE

The model is constructed using Vanilla Mini-Batch Gradient Descent and Softmax Regression. The selected hyperparameters are shown in Table 1.

Model Hyperparameters

Parameter	Value
Learning Rate (η)	0.005
Epochs	100
Batch Size	512
L2 Regularization (λ)	0.01
PCA Components	50
Polynomial Degree	2

OPTIMIZATION FORMULATION

The weight update process uses Vanilla Gradient Descent over mini-batches:

- **Softmax Activation:** For raw logits $Z = XW$, the predicted probabilities $\hat{Y}$ are computed using a numerically stable softmax:

$$\hat{Y} = \frac{\exp(Z - \max(Z))}{\sum \exp(Z - \max(Z))}$$

- **Cross-Entropy Loss:** The loss function measures the divergence between predicted and true class distributions:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log(\hat{y}_{ik} + \epsilon)$$

where $\epsilon = 10^{-15}$ ensures numerical stability.

- **Gradient Computation** (with L_2 regularization λ , excluding the bias row):

$$g_t = \frac{1}{m} X^T (\hat{Y} - Y) + \lambda W_{reg}$$

where W_{reg} is the weight matrix W with the bias row (first row) set to 0 to avoid regularizing the bias terms:

$$W_{reg} = \begin{bmatrix} 0 & 0 & \dots & 0 \\ w_{1,1} & w_{1,2} & \dots & w_{1,K} \\ \vdots & \vdots & \ddots & \vdots \\ w_{D,1} & w_{D,2} & \dots & w_{D,K} \end{bmatrix}$$

- **Update Rule** (with learning rate η):

$$W_{t+1} = W_t - \eta g_t$$

CUSTOM METRIC IMPLEMENTATIONS

ACCURACY

Accuracy is defined as the fraction of exact matches between the true labels Y and predicted labels $\hat{Y}$:

$$Accuracy = \frac{1}{N} \sum_{i=1}^N \mathbf{I}(y_i = \hat{y}_i)$$

ROC-AUC (ONE-VS-REST)

The Receiver Operating Characteristic Area Under the Curve (ROC-AUC) is calculated using a One-vs-Rest approach. For each class c :

- The predicted probabilities are sorted in descending order.
- True Positive Rates (TPR) and False Positive Rates (FPR) are incrementally calculated across all distinct threshold probability points.
- The AUC is estimated using the **Trapezoidal Rule**:

$$AUC_c = \sum_{i=1}^{K-1} \frac{(FPR_{i+1} - FPR_i) \times (TPR_{i+1} + TPR_i)}{2}$$

The final multi-class ROC-AUC is the unweighted mean of all individual AUC_c scores.

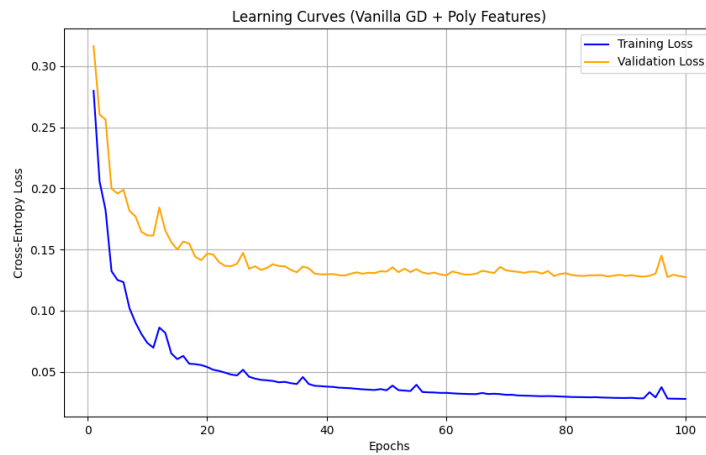
DIAGNOSTICS AND ANALYSIS

K-FOLD CROSS-VALIDATION

A 3-Fold Cross-Validation framework was implemented to reliably estimate the model's generalization capabilities on the polynomial feature space. Each fold was trained for 30 epochs with the same hyperparameters. The CV process yielded a mean validation accuracy of 96.67% ($\pm 0.25\%$), confirming the stability of the selected hyperparameters.

LEARNING CURVES

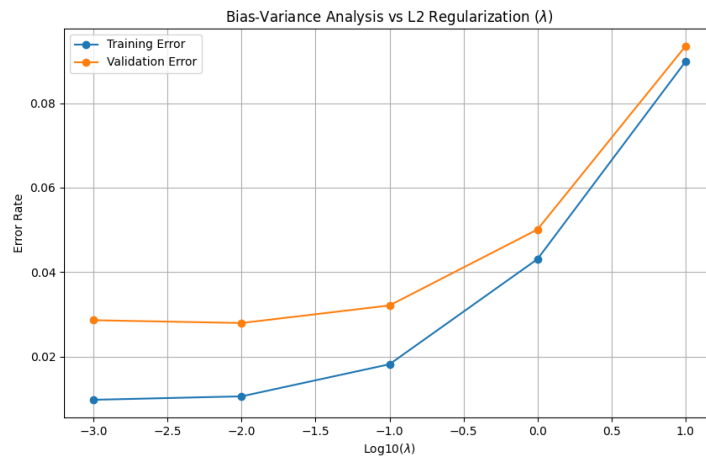
Figure 1 illustrates the cross-entropy loss over 100 epochs for the final model, calculated via the custom training loop. The alignment of training and validation loss indicates stable convergence without severe overfitting.



BIAS-VARIANCE TRADE-OFF

The Bias-Variance trade-off was evaluated by training the model across a logarithmic scale of L2 Regularization values ($\lambda \in \{0.001, 0.01, 0.1, 1.0, 10.0\}$).

- **Low λ (High Variance):** Minimal regularization leads to lower training error but risks increasing validation error.
- **High λ (High Bias):** Excessive regularization over-constrains the weights, causing both errors to spike.



RESULTS

The final model was evaluated on 10,000 unseen test samples.

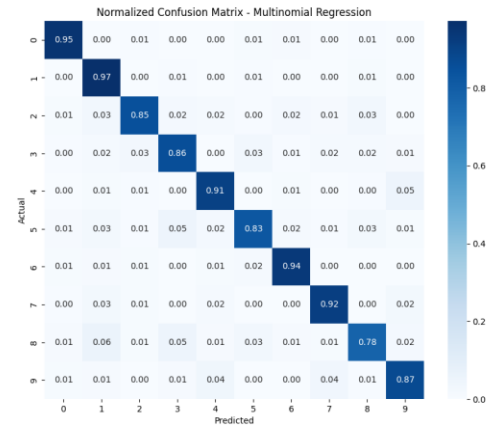
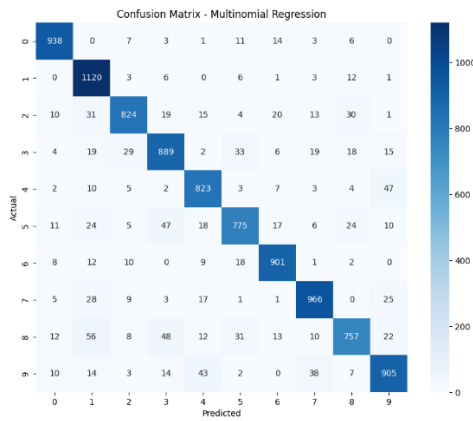
Per Class Performance:

Class (Digit)	Precision	Recall	F1-Score	Support
0	0.9826	0.9766	0.9796	983
1	0.9870	0.9913	0.9892	1152
2	0.9591	0.9700	0.9645	967
3	0.9661	0.9642	0.9652	1034
4	0.9735	0.9746	0.9741	906
5	0.9699	0.9637	0.9668	937
6	0.9734	0.9896	0.9814	961

Class (Digit)	Precision	Recall	F1-Score	Support
7	0.9715	0.9706	0.9711	1055
8	0.9513	0.9484	0.9499	969
9	0.9647	0.9508	0.9577	1036
Accuracy	97.02% (Total Support: 10000)			
Macro Avg	0.9699	0.9700	0.9699	10000
Weighted Avg	0.9702	0.9702	0.9702	10000

Final Model Performance

Metric	Value
Training Accuracy	99.42%
Test Accuracy	97.02%
Multiclass ROC-AUC	0.9971



MODEL: K-NEAREST NEIGHBOURS

MATHEMATICAL FORMULATION:

Given a query $\mathbf{x}^*$ and training set $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$, KNN assigns the label by weighted majority vote over the k nearest neighbours $\mathcal{N}_k(\mathbf{x}^*)$:

$$\hat{y} = \underset{c}{\operatorname{argmax}} \sum_{i \in \mathcal{N}_k(\mathbf{x}^*)} w_i \mathbf{1}[y_i = c], \quad w_i = \frac{1}{d(\mathbf{x}^*, \mathbf{x}_i)} \text{ (distance) or } 1 \text{ (uniform)}$$

Distance metrics: $d_{\text{Eucl}} = \|\mathbf{x} - \mathbf{x}'\|_2$, $d_{\text{Manh}} = \|\mathbf{x} - \mathbf{x}'\|_1$.

MATHEMATICAL FORMULATION OF HYPERPARAMETER TUNING

To find the optimal hyperparameter configuration $\lambda^* = (k, \text{weight}, \text{metric})$ from the search space $\mathcal{A}$, we utilized K -fold stratified cross-validation. Let the training dataset D be partitioned into $K = 5$ disjoint subsets $D_1, D_2, \dots, D_K$.

For a given configuration λ , the model M_λ is trained on $K-1$ folds and evaluated on the held-out validation fold D_i . The optimal configuration maximizes the average cross-validation accuracy:

$$\lambda^* = \operatorname{argmax}_{\lambda \in \Lambda} \frac{1}{K} \sum_{i=1}^K \operatorname{Accuracy} M_{\lambda}(D \setminus D_i, D_i)$$

Where $M_{\lambda}(D \setminus D_i)$ represents the model trained with hyperparameters λ on all data except fold i , and evaluated on the validation fold i .

EVALUATION METRICS (IMPLEMENTED FROM SCRATCH)

For class c , read TP_c , FP_c , FN_c directly from the manually constructed confusion matrix $\mathbf{C} \in \mathbb{Z}^{10 \times 10}$:

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}, \quad F1_c = \frac{2P_c R_c}{P_c + R_c}, \quad \text{Macro-Avg} = \frac{1}{10} \sum_{c=0}^9 (\cdot)$$

No **sklearn.metrics** function was called at any stage, including inside cross-validation.

ENSEMBLE METHODS

Random Forest (Bagging). T trees trained on bootstrap samples; final prediction is majority vote $\hat{y} = \operatorname{mode}\{h_t(\mathbf{x})\}_{t=1}^T$. Averaging uncorrelated trees reduces variance without increasing bias.

Gradient Boosting. Trees added sequentially by fitting the negative loss gradient:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta h_m(\mathbf{x}), \quad h_m = \operatorname{argmin}_h \sum_i \mathcal{L}(y_i, F_{m-1}(\mathbf{x}_i) + h(\mathbf{x}_i))$$

where η is the learning rate. Boosting reduces bias by iteratively correcting residuals of weak learners.

IMPLEMENTATION: FEATURE REPRESENTATIONS

Four feature spaces were evaluated on a balanced 10,000-sample stratified subsample (1,000 per class):

- **Flatten** raw 784-dim pixel vector.
- **PCA** 152 components retaining 95.01% of variance via truncated SVD.
- **HOG** 324-dim Histogram of Oriented Gradients; rotation-robust, edge-aware.
- **CNN** 1,280-dim embeddings from frozen MobileNetV2 (pretrained on ImageNet).

HYPERPARAMETER TUNING 5-FOLD STRATIFIED GRID SEARCH

Search space: $k \in \{1,3,5,7,9,11,15\}$, weights $\in \{\text{uniform, distance}\}$, metric $\in \{\text{euclidean, manhattan}\}$ 28 combinations per feature set, all scored with a custom manual accuracy wrapper.

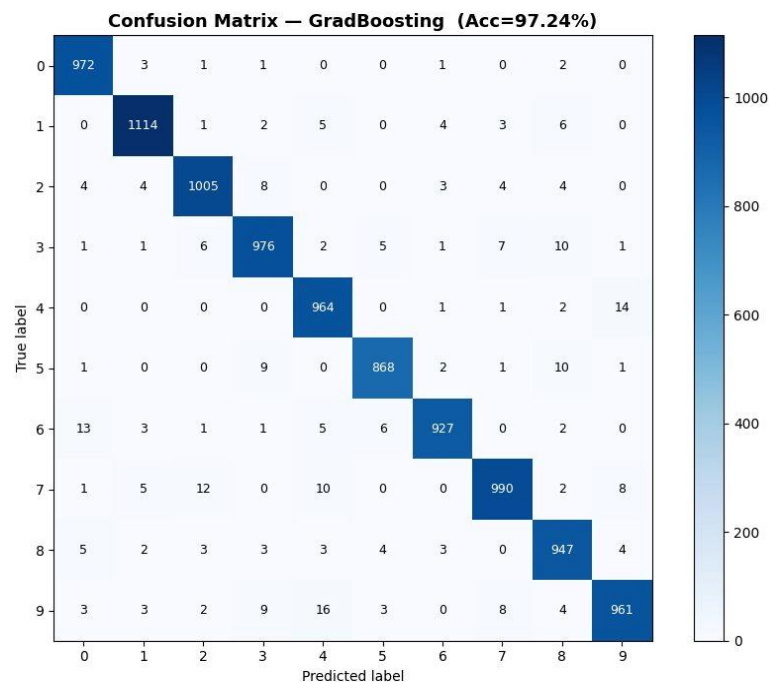
Feature	Best k	Weight	Metric	CV Accuracy
Flatten	5	distance	euclidean	94.46%
PCA	5	distance	euclidean	94.80%
HOG	3	distance	euclidean	95.58%
CNN	7	distance	manhattan	94.28%

EXPERIMENTAL EVALUATION

Model	Accuracy	Precision	Recall	F1-Score
GradBoosting	97.24%	97.24%	97.24%	97.23%
RandomForest	95.68%	95.66%	95.65%	95.65%
KNN-HOG	95.67%	95.75%	95.63%	95.65%
KNN-PCA	95.48%	95.57%	95.43%	95.46%
KNN-Flatten	95.13%	95.25%	95.09%	95.12%
KNN-CNN	94.31%	94.33%	94.18%	94.21%

Final test-set results all six models.

GradBoosting tops all models at **97.24%** accuracy a **+1.57 pp** gain over the best KNN baseline. Random Forest and KNN-HOG are virtually tied at $\approx 95.67\%$. Contrary to expectation, CNN embeddings *underperform* HOG: the 1,280-dimensional space suffers from the curse of dimensionality when only 10,000 training points are available.



IMPROVEMENT STRATEGIES

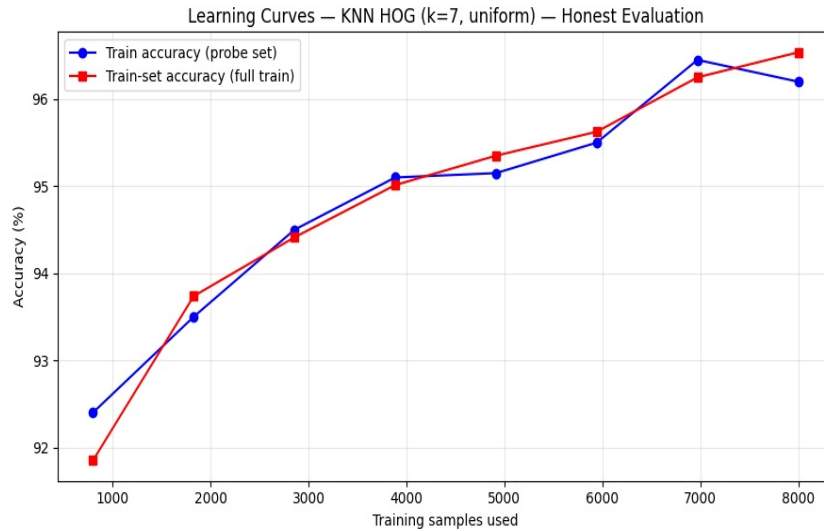
- **Feature Engineering HOG.** Gradient orientation histograms provide edge-aware, scale-robust features; best KNN accuracy 95.67%.
- **Feature Engineering CNN.** MobileNetV2 embeddings transfer deep semantic representations to classical classifiers; CV accuracy 94.28%.
- **Hyperparameter Tuning.** Exhaustive 5-fold stratified grid search identifies the optimal (k , weight, metric) triple per feature set, outperforming any single heuristic choice.
- **Ensemble Bagging (Random Forest).** 300 bootstrap-trained trees reduce variance and achieve 95.68%, matching the best KNN with far lower sensitivity to hyperparameters.
- **Ensemble Boosting (HistGradientBoosting).** $\eta = 0.10$, depth 5, 300 iterations; sequential residual correction yields the top result of **97.24%**.
- **Regularisation via k .** Increasing k smooths decision boundaries, acting as an implicit regulariser the biasvariance tradeoff made explicit.

BIASVARIANCE ANALYSIS

A sweep over $k \in \{1,3,5,7,9,11,15\}$ (HOG) traces the classical U-shaped validation error curve. The optimal $k = 3$ sits at the minimum: $k < 3$ yields high variance (memorising individual neighbours), $k > 7$ yields high bias (distant, irrelevant neighbours dominate).

The learning curve was plotted against a held-out probe set (20% of training data) to avoid the KNN self-memorisation artifact (distance-weighted KNN achieves zero training error by construction). At 8,000 training samples the train/validation gap is **0.34%**, confirming a well-fitted model with negligible overfitting.

OVERFITTING/UNDERFITTING DIAGNOSIS AND LEARNING CURVES



MODEL: KERNEL SVM

DATA PREPARATION

The MNIST dataset (70,000 images, 28×28 pixels) was processed as follows:

- **Train/Validation/Test Split:** 7,500 training samples, 2,500 validation samples, and 14,000 final test samples. Each MNIST image is represented as a flattened vector:

$$x_i \in \mathbb{R}^{784}$$

- **Feature Extraction:** Histogram of Oriented Gradients (HOG) was applied, extracting 324 gradient-based features to reduce dimensionality while capturing spatial structures. The gradient magnitude and orientation are defined as:

$$|\nabla I| = \sqrt{G_x^2 + G_y^2}, \quad \theta = \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

- **Standardization:** Features were strictly scaled to zero mean and unit variance using a manually implemented **StandardScaler**, based on training statistics:

$$z = \frac{x - \mu}{\sigma}$$

- **Label Binarization:** The labels were defined as:

$$y_i = \begin{cases} +1 & \text{digit 7} \\ -1 & \text{all other digits} \end{cases}$$

MODEL OPTIMIZATION FORMULATION

The soft-margin SVM minimizes classification error while maximizing the margin. The primal optimization objective is given by:

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \xi_i$$

subject to $y_i(w^T \phi(x_i) + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$, where the hinge loss penalizes margin violations: $L(y, f(x)) = \max(0, 1 - y \cdot f(x))$.

The model optimization is solved using the dual formulation:

- **Dual Objective:** Maximize the margin by determining Lagrange multipliers α :

$$\max_{\alpha} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

subject to $0 \leq \alpha_i \leq C$ and $\sum_{i=1}^m \alpha_i y_i = 0$.

- **Radial Basis Function (RBF) Kernel:** To model non-linear boundaries, the RBF kernel was used:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

where the kernel scaling parameter is $\gamma = 1/(d \times \text{Var}(X))$.

- **Bias Computation:** The bias b is computed as the average over all margin support vectors (where $0 < \alpha_i < C$):

$$b = \frac{1}{|M|} \sum_{i \in M} \left(y_i - \sum_{j=1}^m \alpha_j y_j K(x_j, x_i) \right)$$

- **Prediction:** The decision function and prediction rule are:

$$f(x) = \sum_{i=1}^m \alpha_i y_i K(x_i, x) + b, \quad \hat{y} = \text{sign}(f(x))$$

CLASS IMBALANCE STRATEGIES

Given the approximate 1:9 ratio of the target class ('7') versus the negative class, standard training models may bias towards predicting the majority class. Three methodologies were applied to combat this:

VARIANT A: CLASS WEIGHTS

The cost parameter C was adjusted differentially for positive (C_+) and negative (C_-) samples based on their inverse frequencies, with base class weights $w_c = n/(k \times n_c)$:

$$C_i = C \times \frac{N_{total}}{K \times N_{class}(y_i)}$$

This equalizes the misclassification penalty during the QP optimization.

VARIANT B: RANDOM UNDERSAMPLING

The majority class ('Not 7') was randomly reduced to match the frequency of the minority class, resulting in a strictly balanced 50/50 dataset size of 1,564 training instances.

VARIANT C: SYNTHETIC MINORITY OVERSAMPLING TECHNIQUE (SMOTE)

A manual SMOTE implementation was employed to synthetically generate new minority instances by interpolating between nearest neighbors within the feature space:

$$x_{new} = x_i + \alpha(x_j - x_i)$$

balancing the dataset to 10,077 total samples.

RESULTS AND DIAGNOSTICS

Final Performance Metrics by Variant on Test Set

Model Variant	Accuracy	Precision	Recall	F1-Score
Variant A (Class Weights)	98.93%	0.9612	0.9349	0.9479
Variant B (Undersampling)	97.59%	0.8246	0.9760	0.8939
Variant C (SMOTE)	98.89%	0.9637	0.9287	0.9459

ONE-VS-ALL OPTIMIZATION ARCHITECTURE

For each digit class $k \in \{0, \dots, 9\}$, a target binary vector $y^{(k)}$ is constructed such that $y_i^{(k)} = 1$ if the instance belongs to class k , and $y_i^{(k)} = -1$ otherwise.

- **Binary QP Optimization:** Each model maximizes the dual margin objective:

$$\max_{\alpha^{(k)}} \sum_{i=1}^m \alpha_i^{(k)} - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i^{(k)} \alpha_j^{(k)} y_i^{(k)} y_j^{(k)} K(x_i, x_j)$$

subject to $0 \leq \alpha_i^{(k)} \leq C$ and $\sum_{i=1}^m \alpha_i^{(k)} y_i^{(k)} = 0$.

- **Decision Function Formulation:** For an unseen sample x , each binary classifier outputs a confidence score proportional to its geometric distance from the hyperplane:

$$f_k(x) = \sum_{i \in SV^{(k)}} \alpha_i^{(k)} y_i^{(k)} \exp(-\gamma \|x_i - x\|^2) + b^{(k)}$$

- **Multiclass Aggregation:** The final predicted class $\hat{y}$ is selected using the *argmax* rule:

$$\hat{y} = \arg \max_{k \in \{0, \dots, 9\}} f_k(x)$$

HYPERPARAMETER TUNING WITH CROSS-VALIDATION

Grid Search combined with 3-Fold Cross-Validation was used to determine the optimal hyperparameters.

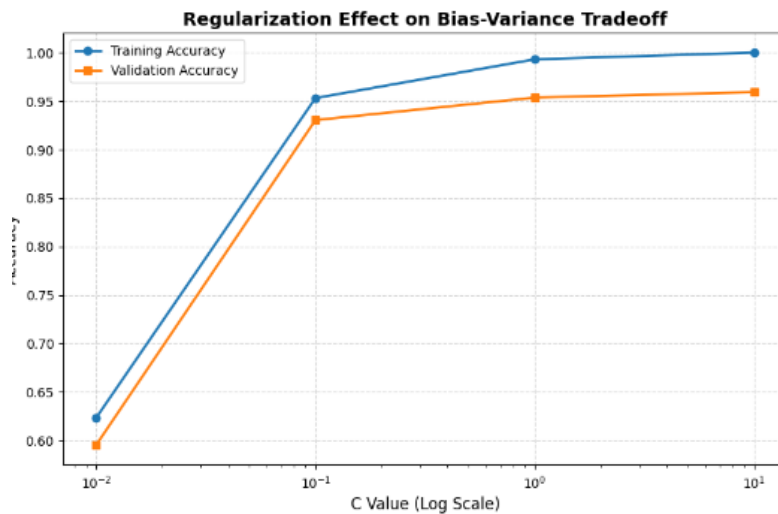
- Tested C values: {0.1, 1, 10}
- Tested gamma values: {scale, 0.01}

BIAS-VARIANCE ANALYSIS

Regularization strength is controlled through the penalty parameter C . Smaller C introduces stronger regularization (higher bias, lower variance); larger C weakens regularization (lower bias, higher variance):

- **Small C (High Bias):** The model under-fits; both training and validation accuracy are low.
- **Large C (High Variance):** The model over-fits; training accuracy is high while validation accuracy drops.

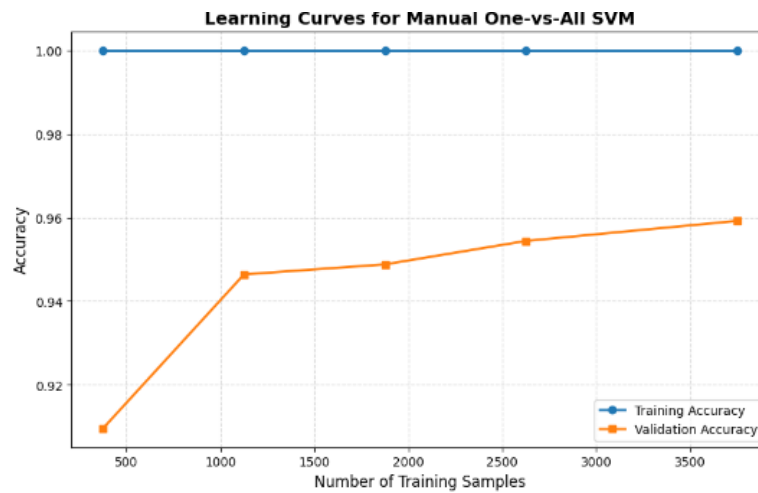
Learning curves were generated by retraining the best configuration on increasing fractions of the training set and monitoring the train/validation accuracy gap.



OVERFITTING/UNDERFITTING DIAGNOSIS AND LEARNING CURVES

Learning curves were generated using different training-set sizes.

- A large gap between training and validation accuracy indicates overfitting, while low converging accuracies indicate underfitting.



RESULTS

FINAL TEST EVALUATION

Using Best Parameters:

C = 10

Gamma = scale

Predicting Final Test Set...

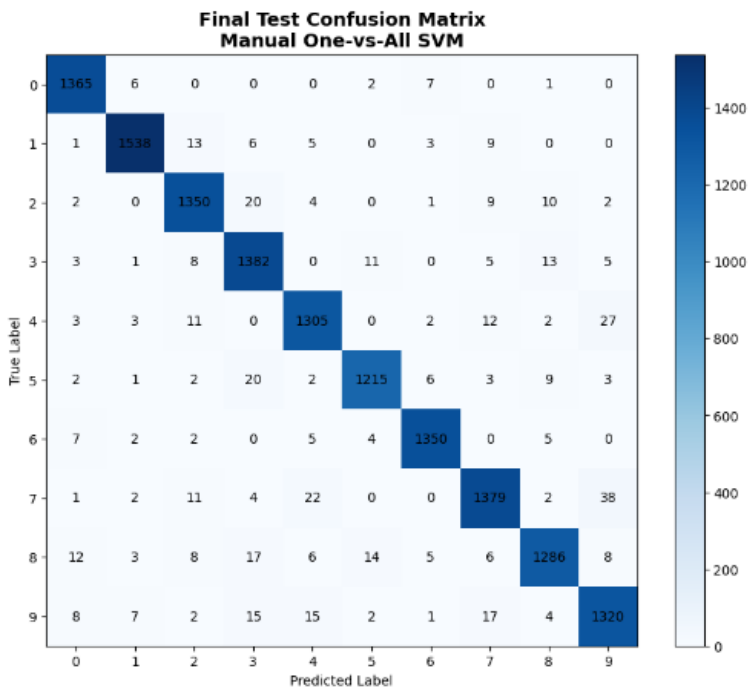
Final Test Accuracy: 0.9636

CLASSIFICATION REPORT

Class	Precision	Recall	F1-Score
0	0.9722	0.9884	0.9803
1	0.9840	0.9765	0.9802
2	0.9595	0.9657	0.9626
3	0.9440	0.9678	0.9557
4	0.9567	0.9560	0.9564
5	0.9736	0.9620	0.9677
6	0.9818	0.9818	0.9818
7	0.9576	0.9452	0.9514
8	0.9655	0.9421	0.9537
9	0.9408	0.9490	0.9449

MACRO AVERAGES

Precision: 0.9636
Recall: 0.9634
F1-Score: 0.9635



MODEL: MULTICLASS LINEAR SVM

DATA PREPARATION

The MNIST dataset (70,000 images of handwritten digits, 784 pixels each) was processed through the following pipeline:

- **Train/Validation/Test Split:** 80% training, 20% validation from the standard MNIST 60,000-sample training set; 10,000 samples reserved for testing. Splits are stratified to maintain class balance.
- **Normalization:** Pixel values normalized to the range $[0,1]$.
- **Feature Pipelines:** Four feature representations were evaluated: Flattened pixels (784-D), PCA projections (100-D), HOG descriptors, and MobileNetV2 CNN embeddings (1280-D).

MATHEMATICAL FORMULATION

For each image x , the feature pipeline produces $\phi(x) \in \mathbb{R}^d$. The multiclass Linear SVM learns one weight vector per class. With $K = 10$ classes, the score for class k is:

$$s_k(x) = w_k^T \phi(x) + b_k$$

The prediction rule is:

$$\hat{y} = \operatorname{argmax}_k s_k(x)$$

For a sample (x_i, y_i) , the Crammer-Singer multiclass hinge loss is:

$$L_i = \sum_{k \neq y_i} \max(0, s_k(x_i) - s_{y_i}(x_i) + \Delta)$$

The regularized objective function optimized during training is:

$$J(W, b) = \frac{1}{2} \|W\|_F^2 + C \cdot \frac{1}{N} \sum_{i=1}^N L_i$$

where the first term is L2 (Frobenius norm) regularization and $C > 0$ controls the trade-off between margin maximization and classification error. A larger C penalizes classification errors more strongly; a smaller C increases regularization.

OPTIMIZATION WITH ADAM

The objective is minimized using the **Adam** optimizer with bias-corrected moment estimates. For each mini-batch the subgradient of the hinge loss is computed analytically.

For each sample i in the batch, define the coefficient matrix $A \in \mathbb{R}^{B \times K}$ as:

$$A_{ik} = \begin{cases} +1 & \text{if } k \neq y_i \text{ and } s_k(x_i) - s_{y_i}(x_i) + \Delta > 0 \\ -\sum_{k' \neq y_i} \mathbf{1}[\text{margin}_{ik'} > 0] & \text{if } k = y_i \\ 0 & \text{otherwise} \end{cases}$$

The gradients with respect to W and b are:

$$\nabla_W J = W + \frac{C}{B} X_{\text{batch}}^T A, \quad \nabla_b J = \frac{C}{B} \sum_{i=1}^B A_{i,:}$$

The Adam update rule at step t is:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \end{aligned}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_t = \theta_{t-1} - \frac{\eta_t}{\sqrt{\hat{v}_t} + \varepsilon} \hat{m}_t$$

where $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, and the learning rate decays multiplicatively each epoch: $\eta_{e+1} = 0.95 \eta_e$.

Training uses early stopping: if validation loss does not improve for 5 consecutive epochs, training halts and the best-seen weights are restored.

FEATURE PIPELINES

Four feature spaces were evaluated:

- **Flatten:** Raw 28×28 pixel values reshaped into a 784-dimensional vector.
- **PCA:** Flattened pixels projected to 100 principal components fitted on training data only.
- **HOG:** Histogram of Oriented Gradients descriptors; captures local edge and gradient structure.
- **CNN:** MobileNetV2 backbone (ImageNet weights, global average pooling) maps each image to a 1280-dimensional embedding. Features are pre-extracted and cached.

HYPERPARAMETER TUNING WITH CROSS-VALIDATION

A 3-Fold Stratified Cross-Validation grid search was applied on a stratified 9,000-sample subset of the training data. The search space was:

- Feature modes: {flatten, pca, hog}
- $C \in \{0.1, 1.0, 5.0, 10.0, 20.0, 50.0\}$
- Learning rate: 0.01 (flatten/pca), 0.02 (hog)
- Epochs: 12 per fold

The optimal configuration maximizes mean validation macro F1:

$$\lambda^* = \operatorname{argmax}_{\lambda \in \Lambda} \frac{1}{K} \sum_{i=1}^K \operatorname{MacroF1}(M_\lambda(D \setminus D_i), D_i)$$

BIAS-VARIANCE ANALYSIS

Regularization strength is controlled through the penalty parameter C . Smaller C introduces stronger regularization (higher bias, lower variance); larger C weakens regularization (lower bias, higher variance):

- **Small C (High Bias):** The model under-fits; both training and validation accuracy are low.
- **Large C (High Variance):** The model over-fits; training accuracy is high while validation accuracy drops.

Learning curves were generated by retraining the best configuration on increasing fractions of the training set and monitoring the train/validation accuracy gap.

ENSEMBLE BASELINES

Two ensemble methods were trained on PCA features (100 components) for comparison:

- **Random Forest (Bagging):** T trees trained on bootstrap samples; final prediction by majority vote $\hat{y} = \operatorname{mode}\{h_t(x)\}_{t=1}^T$. Reduces variance without increasing bias.

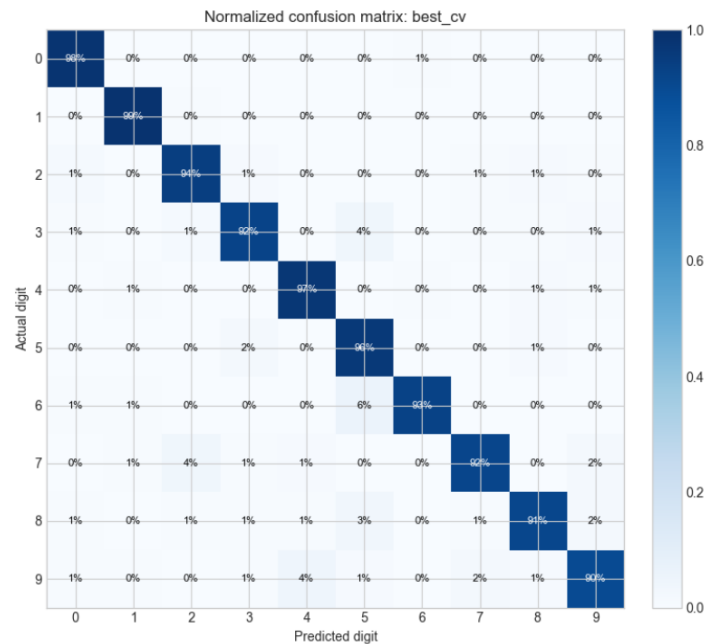
- **AdaBoost:** Sequentially fits weak learners (Decision Trees) by re-weighting misclassified samples. Reduces bias through iterative correction.

RESULTS

Classification Report – best_cv on test set

	precision	recall	f1-score	support
0	0.95	0.98	0.97	980
1	0.97	0.99	0.98	1135
2	0.93	0.94	0.94	1032
3	0.95	0.92	0.94	1010
4	0.93	0.97	0.95	982
5	0.86	0.96	0.91	892
6	0.98	0.93	0.95	958
7	0.96	0.92	0.94	1028
8	0.94	0.91	0.93	974
9	0.94	0.90	0.92	1009
accuracy	0.94	0.94	0.94	10000
macro avg	0.94	0.94	0.94	10000
weighted avg	0.94	0.94	0.94	10000

	precision	recall	f1-score	support
0	0.9545	0.9837	0.9688	980.0
1	0.9739	0.9877	0.9808	1135.0
2	0.9329	0.9428	0.9378	1032.0
3	0.9530	0.9228	0.9376	1010.0
4	0.9343	0.9705	0.9520	982.0
5	0.8590	0.9630	0.9080	892.0
6	0.9791	0.9269	0.9523	958.0
7	0.9574	0.9173	0.9369	1028.0
8	0.9447	0.9127	0.9285	974.0
9	0.9368	0.8959	0.9159	1009.0
accuracy	0.9426	0.9426	0.9426	10000.0
macro avg	0.9425	0.9423	0.9419	10000.0
weighted avg	0.9437	0.9426	0.9426	10000.0



GITHUB LINK

<https://github.com/AbdulrhmanHammouda/mnist-image-classification>